LSTM-ARO 簡報｜第 1–11 頁 研究筆記與講稿

> 依據：新版網頁簡報（共 27 頁）與論文原文 §1–§3.6、§4.1、Fig. 2、Tables 1–4。
> 建議時長：第 1–11 頁約 11 分鐘（30 分鐘論文報告中的前段）。

一、核心觀念速懂（被問到要能回答）

1. 這篇論文在做什麼（第 1–3 頁）

一句話：**用 ARO（人工兔群最佳化）自動挑選 LSTM 的架構與訓練設定，再用挑出來的 LSTM 預測股價。**

	ARO（外層）	LSTM（內層）
負責	搜尋超參數：層數、神經元數、Dropout、優化器、學習率	讀過去 20 天價格，預測隔天價格
怎麼學	一群「兔子」各代表一組超參數，依表現好壞更新位置	用訓練資料做反向傳播，學習權重
重複次數	50 隻兔子 × 50 代 = 2,500 組設定	每組設定訓練一次

- **ARO 不學 LSTM 的權重**，它只決定「要建一個什麼樣的 LSTM」。
- 適應度（兔子好不好）：論文 Fig. 2 只寫「用測試資料計算 accuracy and error」回傳給 ARO，**沒寫明是哪個指標**，不要說成「用 MSE」。

2. 相關研究（第 4 頁）

方向	代表方法	限制或角色
傳統時間序列	ARIMA	以過去數值與誤差做線性組合，難描述高度非線性的股價關係
深度時間模型	LSTM、CNN-LSTM	能學時間相依性，但層數與參數設定會影響結果
混合最佳化	GA 等演化演算法	把調參變成搜尋問題，減少手動試誤

本論文的切入點：改用 2022 年提出的 ARO 搜尋深度 LSTM 的超參數，並和 GA 調參比較。

3. 資料（第 5 頁）

- 來源：Yahoo Finance；道瓊工業指數（DJIA）30 檔成分股；2018-01-01 至 2022-12-31，共 5 年日資料。
- 樣本：**前 20 天價格 → 預測第 21 天**，視窗每次往後移 1 天，產生下一筆樣本。
- 切分：80% 訓練、20% 測試。

- 注意（論文沒寫清楚的地方）：論文只寫「prices」，沒指明是收盤價；也沒寫 80/20 是否依時間先後切分。簡報依時間序列的慣例畫成「前 80%、後 20%」。(N, 20, 1) 是 Keras LSTM 的標準輸入形狀，論文沒有寫出來。

4. ANN → RNN → LSTM（第 6 頁）

模型	怎麼看待時間	問題
ANN	把 20 天價格當成 20 個獨立的輸入，不建模先後順序	看不出「走勢」
RNN	逐日讀入，把隱藏狀態傳給下一天	梯度跨很多步連乘，容易消失，記不住較久以前的資訊
LSTM	多了一條 cell state，用門控決定保留與寫入	以加法更新記憶，梯度較不易消失

5. LSTM Cell 與四個門（第 7–9 頁，論文 Eqs. 1–5）

名稱	公式	作用
遺忘門 f_t	$\sigma(W_f[h_{t-1}, x_t] + b_f)$	舊記憶保留多少（0–1）
輸入門 i_t	$\sigma(W_i[h_{t-1}, x_t] + b_i)$	新資訊寫入多少（0–1）
候選記憶 $\tilde{C}_t$	$\tanh(W_c[h_{t-1}, x_t] + b_c)$	這一步想寫入的內容（−1 到 1）
輸出門 o_t	$\sigma(W_o[h_{t-1}, x_t] + b_o)$	對外輸出多少（0–1）
新記憶 C_t	$f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$	保留的舊記憶 + 寫入的新資訊
輸出 h_t	$o_t \odot \tanh(C_t)$	傳給下一天，也傳給 Dense 層

第 9 頁算例：$f_t = 0.8$、$C_{t-1} = 0.5$、$i_t = 0.6$、$\tilde{C}_t = 0.7$、$o_t = 0.5$ → $C_t = 0.40 + 0.42 = 0.82$；$h_t = 0.5 \times \tanh(0.82) \approx 0.5 \times 0.675 \approx 0.338$。

6. 沿時間展開（第 10 頁）

- 同一個 LSTM Cell、**同一組權重**，依序處理第 1 天到第 20 天；每一步把 C_t、h_t 傳給下一天。
- 讀完第 20 天後，用 h_{20} 經過 Dense 層輸出第 21 天的預測 $\hat{y}_{21}$。
- 「展開 20 步」是時間方向；「LSTM1D / 2D / 3D」是把層往上疊，是深度方向，兩者不同。

7. 對照模型（第 11 頁，論文 Tables 1–5）

模型	架構
ANN	Input(20) → Dense(10) → Dense(10) → Dropout(0.5) → Dense(1)
LSTM1D	Input(20) → LSTM(10) → Dense(10) → Dropout(0.5) → Dense(1)
LSTM2D	Input(20) → LSTM(10) × 2 → Dense(10) → Dropout(0.5) → Dense(1)
LSTM3D	Input(20) → LSTM(10) × 3 → Dense(10) → Dropout(0.5) → Dense(1)
LSTM-GA / ARO	層數 1–3、各層神經元、Dense 神經元、Dropout、優化器、學習率由演算法搜尋

「1D / 2D / 3D」的 D 指 LSTM 層數，不是一維、二維卷積。

二、講稿（約 11 分鐘）

Slide 1 | 封面（約 30 秒）

大家好，我們今天報告的論文是 Gülmez 在 2023 年發表於 Expert Systems with Applications 的〈Stock price prediction with optimized deep LSTM network with artificial rabbits optimization algorithm〉，中文是「以人工兔群最佳化演算法優化深度 LSTM 的股價預測」。

Slide 2 | 研究問題（約 1 分鐘）

這篇論文想解決兩個問題。第一，股價有時間依賴，下一天的價格和前面好幾天的走勢有關，所以需要能處理序列的模型。第二，LSTM 雖然適合處理序列，但層數、神經元數、學習率這些設定會影響準確度，而且組合很多，靠人工一個個試很沒效率。所以作者的想法是：讓 ARO 這個群體式的搜尋演算法，自動找出預測誤差最低的 LSTM 架構與訓練設定。

Slide 3 | 整體架構（約 1 分 15 秒）

整個系統分成內外兩層。外層是 ARO，負責提出一組超參數，包括 LSTM 要幾層、每層幾個神經元、Dropout、優化器和學習率。內層是 Deep LSTM，依照這組設定建模、訓練，讀過去 20 天的股價、預測隔天的價格，再把在測試資料上的準確度與誤差回傳給 ARO 當作適應度。ARO 根據適應度更新，這樣來回 50 代，最後留下誤差最低的那組設定，作為最終的預測模型。重點是：ARO 是調參器，LSTM 才是預測器；ARO 並不學 LSTM 的權重。

Slide 4 | 相關研究（約 1 分鐘）

過去的研究大致有三個方向。傳統時間序列像 ARIMA，用過去的數值和誤差做線性組合，但股價的關係往往不是線性的。深度時間模型像 LSTM、CNN-LSTM 能學到時間相依性，但效果很依賴架構和參數設定。第三類是混合最佳化，用 GA 這類演化演算法來搜尋超參數，減少人工試誤。這篇論文的切入點，就是改用較新的 ARO 來搜尋深度 LSTM 的超參數，並且和 GA 調參做比較。

Slide 5 | 資料處理（約 1 分 10 秒）

資料來自 Yahoo Finance，是道瓊 30 檔成分股從 2018 到 2022 年、共五年的日資料。作者用滑動視窗把價格序列變成監督式學習的樣本：前 20 天的價格當輸入，第 21 天當預測目標；視窗每次往後移一天，就得到下一筆樣本。資料的 80% 用來訓練、20% 用來測試。輸入到 LSTM 時，每筆樣本是 20 個時間步、每步 1 個特徵。

Slide 6 | ANN、RNN、LSTM（約 1 分 10 秒）

為什麼要用 LSTM？先看一般的 ANN，它把 20 天的價格當成 20 個獨立的輸入，不會考慮先後順序，論文把它當作對照組之一。RNN 會逐日讀入資料，並把隱藏狀態傳給下一天，所以有時序性；但梯度要跨很多步連乘，容易消失，很難記住較久以前的資訊。LSTM 多了一條叫 cell state 的記憶通道，用三個門控制要保留和寫入什麼，而且記憶是用加法更新的，梯度比較不容易消失。這也是論文預測的核心模型，而它要疊幾層，就是交給 ARO 決定。

Slide 7 | LSTM Cell 的資訊流（約 1 分 10 秒）

這張圖是 LSTM 在一個時間步裡做的事。上面是記憶的路徑：舊記憶 C_{t-1} 先乘上遺忘門，決定留下多少，再加上新資訊，得到新記憶 C_t；新記憶經過 tanh、再乘上輸出門，得到這一步的輸出 h_t。下面四張卡是這些門：遺忘門決定舊記憶保留多少，輸入門決定寫入多少，候選記憶是這一步想寫的內容，輸出門決定對外輸出多少。它們都讀同一組輸入，也就是前一步的輸出 h_{t-1} 和今天的資料 x_t。

Slide 8 | 每個門怎麼算（約 1 分鐘）

每個門的計算方式都一樣：把 h_{t-1} 和 x_t 接在一起，乘上自己的權重、加上偏差，再通過激活函數。遺忘門、輸入門、輸出門用 sigmoid，輸出介於 0 到 1，可以理解成「通過多少比例」；候選記憶用 tanh，輸出介於 −1 到 1，代表要寫入的內容。這四組權重 W 和偏差 b 都是訓練時學出來的，也就是論文的式 1 到式 5。

Slide 9 | 帶入數字（約 1 分鐘）

我們帶一組數字看看。假設遺忘門 0.8、舊記憶 0.5，保留下來的是 0.4；輸入門 0.6、候選記憶 0.7，寫入的是 0.42；兩者相加，新記憶就是 0.82。輸出門 0.5，乘上 tanh(0.82) 約 0.675，輸出就是約 0.338。可以看到 cell state 是更新後的記憶，hidden state 則是經過輸出門篩選後真正輸出的部分。

Slide 10 | 沿時間展開（約 50 秒）

要注意，一個 LSTM 層不是只跑一次，而是用同一個 cell、同一組權重，依序處理第 1 天到第 20 天，每一步都把記憶和輸出傳給下一天。讀完第 20 天之後，再用最後的輸出 h_{20} 經過輸出層，預測第 21 天的價格。

Slide 11 | 對照模型（約 1 分鐘）

最後介紹論文比較的模型。LSTM1D、2D、3D 是固定架構，D 指的是 LSTM 的層數，分別疊 1、2、3 層，每層 10 個神經元，後面接 Dense、Dropout 0.5，最後輸出 1 個值。另外還有 ANN 作為基準。而 LSTM-GA 和 LSTM-ARO 的層數與超參數，則是由演算法搜尋決定。接下來由下一位同學說明 GA 和 ARO 是怎麼搜尋的。

三、這段可能被問的問題

Q1：為什麼用 LSTM，不用 ARIMA？ ARIMA 是線性模型，假設序列平穩、關係是線性的；股價常受多種因素交互影響，關係不是固定比例。LSTM 可以學非線性的時間關係。但論文沒有拿 ARIMA 當對照組，所以這篇無法直接證明 LSTM 比 ARIMA 好。

Q2：ARO 是在學 LSTM 的權重嗎？ 不是。ARO 只決定架構與訓練設定（層數、神經元、Dropout、優化器、學習率），權重仍由反向傳播在訓練時學出來。

Q3：為什麼視窗是 20 天？ 論文沒有說明理由。20 個交易日大約是一個月；視窗長度本身也可以當成超參數來搜尋，但論文沒有這麼做。

Q4：LSTM 為什麼比較不容易梯度消失？ RNN 的梯度每往前一步都要乘上權重矩陣，連乘很多次會趨近 0。LSTM 的 cell state 是用「保留比例 × 舊記憶 + 新資訊」更新，當遺忘門接近 1 時，梯度可以沿著這條加法路徑傳遞，比較不會被反覆縮小。這是「緩解」，不是完全解決。

Q5：資料有沒有做正規化？ 論文沒有寫。一般會把價格縮放到 0–1，而且縮放用的最大、最小值只能從訓練集計算，否則會把測試期的資訊洩漏進訓練。

Q6：LSTM1D 的 1D 是一維卷積嗎？ 不是，D 指 LSTM 的層數。LSTM1D 是 1 層 LSTM，LSTM3D 是 3 層 LSTM。

Q7：30 檔股票是共用一個模型，還是每檔各一個？ 論文沒有明確說明。結果表是逐檔列出誤差，Table 11 也有「每檔預測時間」，看起來是每檔各自預測，但無法從論文確認是否每檔各自調參。

參考文獻

Gülmez, B. (2023). Stock price prediction with optimized deep LSTM network with artificial rabbits optimization algorithm. *Expert Systems with Applications*, 227, 120346. https://doi.org/10.1016/j.eswa.2023.120346